

Gymnázium J. K. Tyla



Agentické inženýrství ve vývoji softwaru

Návrh a implementace DarkFactory

ODBORNÁ PRÁCE

Autor práce: Patrik Marius, 4.D

Vedoucí práce: Michal Dočekal

Prohlášení

Prohlašuji, že jsem tuto studentskou odbornou práci vypracoval samostatně pod dohledem vedoucího uvedeného na první straně. Všechny použité zdroje jsou uvedeny v seznamu zdrojů a informace z nich získané jsou v textu řádně označeny odkazem na zdroj. Souhlasím s tím, aby tištěná forma práce byla uchována na Gymnáziu J. K. Tyla a tam používána jako tištěný zdroj např. pro další studentské práce či pro prezentaci vzdělávání na GJKT.

V Hradci Králové dne

Podpis autora práce:

.....

Anotace

Práce se zabývá současným agentickým inženýrstvím ve vývoji softwaru. S rostoucími schopnostmi a adopcí generativní AI se rozšiřují možnosti coding agentů, jejich praktické využití je však stále méně rozšířené než běžné používání chatbotů. Cílem práce je popsat a systematizovat principy současného agentického inženýrství a na systému DarkFactory ukázat, co jejich propojení umožňuje v praxi.

Teoretická část vysvětluje vztah mezi jazykovým modelem, agentem a harness vrstvou a shrnuje práci s kontextem, nástroji, trvalým stavem, specifikací, orchestrací, programovým ověřováním a lidskými rozhodovacími body. Popisuje také současnou podobu softwarové továrny, v níž jsou coding agenti začleněni do běžného repozitářového workflow.

Praktická část má podobu inženýrské případové studie DarkFactory, záměrně jednoduché agentické softwarové továrny nativně postavené na GitHubu. Systém byl implementován pomocí komerčních coding agentů a slouží jako bootstrap: již dokáže agenty sám spouštět a řídit a vytváří základ pro další rozvoj stejným procesem.

Studie ukazuje, že praktické agentické workflow vzniká kombinací modelového úsudku s trvalým stavem mimo model, programovou orchestrací, deterministickými kontrolami a lidskými branami. Současně odhaluje omezení počáteční implementace, zejména u modelové revize, opakování verifikace a změn schváleného rozsahu. Přínosem práce je systematizace těchto principů a ukázka jejich konkrétní realizace v jednom produkčním vývojovém procesu.

Klíčová slova: coding agenti; harness; softwarová továrna; orchestrace; generativní AI

Abstract

The thesis addresses contemporary agentic engineering in software development. As the capabilities and adoption of generative AI grow, the possibilities of coding agents also expand, but their practical use remains less widespread than ordinary chatbot use. The aim of the thesis is to describe and systematize the principles of contemporary agentic engineering and, using DarkFactory, show what their combination enables in practice.

The theoretical part explains the relationship between the language model, the agent, and the harness layer and summarizes work with context, tools, durable state, specification, orchestration, programmatic verification, and human decision points. It also describes the contemporary form of the software factory, in which coding agents are integrated into ordinary repository workflows.

The practical part takes the form of an engineering case study of DarkFactory, a deliberately simple agentic software factory built natively on GitHub. The system was implemented using commercial coding agents and serves as a bootstrap: it can already launch and manage agents itself and creates a foundation for further development using the same process.

The study shows that a practical agentic workflow arises from combining model judgment with durable state outside the model, programmatic orchestration, deterministic checks, and human gates. At the same time, it reveals limitations of the initial implementation, particularly in model-based review, repeated verification, and changes to the approved scope. The contribution of the thesis is the systematization of these principles and a demonstration of their concrete implementation in a single production development process.

Keywords: coding agents; harness; software factory; orchestration; generative AI

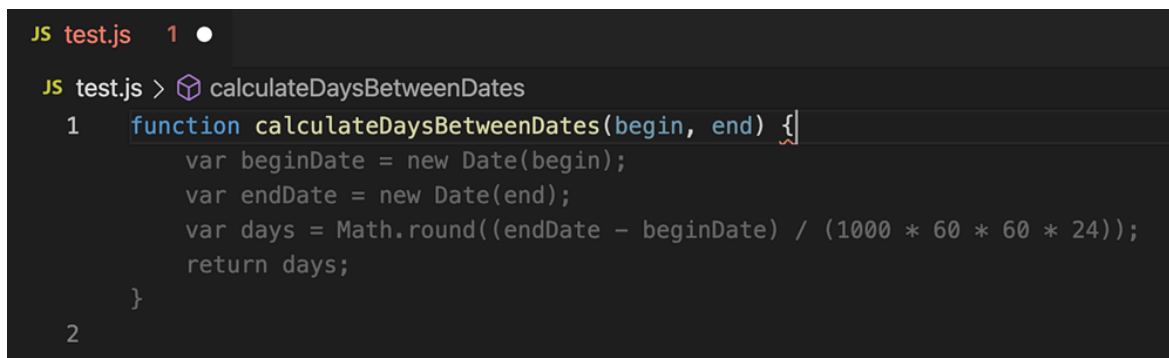
Obsah

Prohlášení	2
1 Úvod	6
1.1 Motivace: Vývoj a adopce generativní AI	6
1.2 Cíl, výzkumná otázka a vymezení	9
1.3 Terminologie	10
2 Teoretická část	11
2.1 Agent: Co to je a jak funguje	11
2.1.1 Jazykový model (LLM)	11
2.1.2 Smyčka (Loop)	12
2.1.3 Nástroje (Tools)	13
2.1.4 Kontext (Context)	14
2.2 Agentické inženýrství	15
2.2.1 Specifikace (Specification)	15
2.2.2 Orchestrace (Orchestration)	17
2.3 Softwareová továrna	21
3 Praktická část	23
3.1 Metodika	23
3.2 Návrh systému DarkFactory	24
3.3 Architektura systému	26
3.4 Průchod požadavku systémem	28
3.5 Implementace pomocí coding agentů	30
4 Zjištění a diskuse	31
5 Závěr	33
Seznam zdrojů	35
Seznam součástí textu	39

1 Úvod

1.1 Motivace: Vývoj a adopce generativní AI

Od svého vzniku se nástroje založené na jazykových modelech neustále zlepšují a roste také jejich adopce. Jednou z prvních široce používaných forem ve vývoji softwaru bylo doplňování kódu přímo v editoru (1), jehož příklad ukazuje Obrázek 1.

A screenshot of a code editor with a dark theme. The top bar shows 'JS test.js' and a tab indicator. The main area displays a prompt 'JS test.js > calculateDaysBetweenDates' followed by a function definition. The function is partially typed, and the AI model has suggested the closing curly brace '}' for the first line of the function body. The line numbers 1 and 2 are visible on the left margin.

```
JS test.js 1 ●  
  
JS test.js > calculateDaysBetweenDates  
1  function calculateDaysBetweenDates(begin, end) {  
    var beginDate = new Date(begin);  
    var endDate = new Date(end);  
    var days = Math.round((endDate - beginDate) / (1000 * 60 * 60 * 24));  
    return days;  
    }  
2
```

Obrázek 1: Doplňování kódu přímo v editoru: model navrhuje pokračování řádku, které člověk přijme nebo odmítne (1).

Potom přišly konverzační chatboty, v nichž model sestavuje odpověď, ale nástroje mu zpravidla nebyly k dispozici, takže i nadále všechno provedl uživatel. Tento způsob použití ilustruje Obrázek 2.

P

A 59-year-old female comes to your office for routine examination. On a panoramic radiograph, you noticed some incidental findings. Please provide your diagnosis based on the radiograph below.

1. Florid cemento-osseous dysplasia
2. Secondary hyperparathyroidism
3. Fibrous dysplasia
4. Hypercementosis



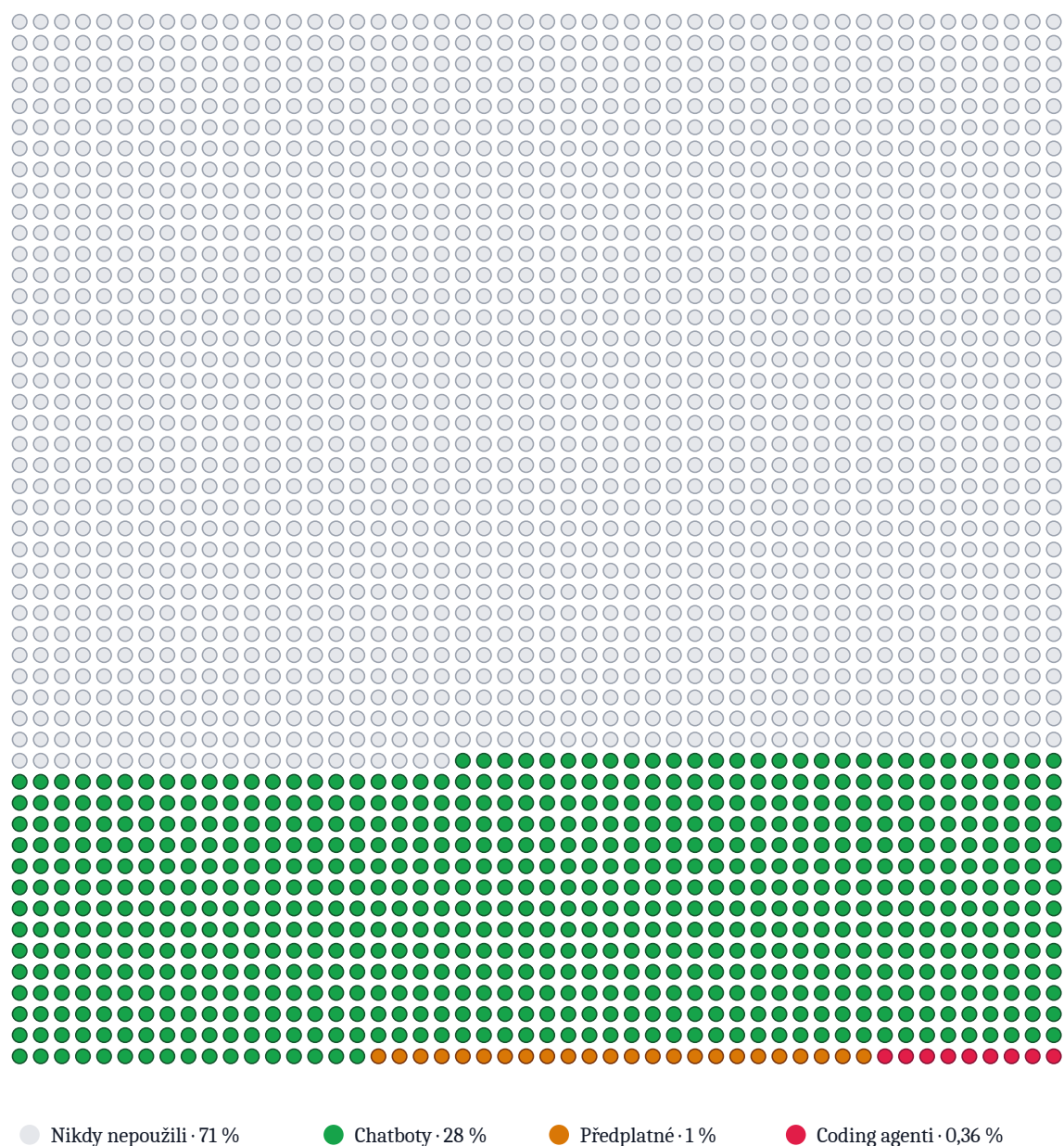
As an AI language model, I'm unable to directly view or interpret images. However, I can provide you with a brief description of the conditions you mentioned to help you understand them better.

1. Florid cemento-osseous dysplasia: This is a benign condition that affects the jawbones, specifically the cementum and surrounding bone. It typically appears as multiple radiopaque masses in the jawbones and is often asymptomatic.
2. Secondary hyperparathyroidism: This condition occurs due to chronic kidney disease or vitamin D deficiency, leading to abnormal parathyroid gland function. It can result in increased bone resorption and subsequent radiographic changes.
3. Fibrous dysplasia: Fibrous dysplasia is a bone disorder where normal bone is replaced by fibrous tissue. It can present as an incidental finding on radiographs, showing a radiolucent and ground-glass appearance in affected areas.
4. Hypercementosis: Hypercementosis is a condition characterized by excessive deposition of cementum around the roots of teeth. It can appear as a radiopaque enlargement around the roots of affected teeth.

Without visual examination and additional information, it is challenging to provide an accurate diagnosis based solely on a radiograph. It is recommended to consult with a qualified dental or medical professional who can evaluate the patient's clinical history, symptoms, and perform a physical examination to provide an accurate diagnosis.

Obrázek 2: Chatbot odpovídá textem a práci nechává člověku: místo aby snímek posoudil, přiznává, že obrázky neumí, a je to uživatel, kdo musí dojít k závěru. Snímek z bezplatné verze ChatGPT, převzato z (2).

Další posun představují *agenti* (3; 4; 5), kteří mohou získat přístup k souborům, příkazům a běhovému prostředí. Model tak již nejen navrhuje výsledek, ale může prostřednictvím nástrojů sám provádět jednotlivé kroky práce.



Obrázek 3: Odhad rozdělení uživatelů generativní AI podle typu (6).

Podle jednoho zveřejněného odhadu používá bezplatné AI chatboty přibližně 28 % světové populace, zatímco pravidelní uživatelé coding agentů tvoří přibližně 0,36 % (6) Obrázek 3. Agentické nástroje jsou tedy stále výrazně méně rozšířené než běžné konverzační použití generativní AI. Motivací této práce je proto ukázat, čeho lze s těmito nástroji dosáhnout při použití současných postupů agentického inženýrství.

1.2 Cíl, výzkumná otázka a vymezení

Cílem práce je popsat a systematizovat principy současného agentického inženýrství ve vývoji softwaru a na systému DarkFactory ukázat, co jejich propojení umožňuje v praxi. Pozornost je přitom věnována také způsobu, jakým lze coding agenty začlenit přímo do běžného repozitářového workflow. Takové propojení agentů s existujícími procesy vývoje se již používá v produkčním softwarovém inženýrství a představuje současnou podobu myšlenky softwarové továrny (7; 8).

Výzkumná otázka práce zní: *Jaké architektonické a procesní principy se opakují v současném agentickém vývoji softwaru a jak jsou realizovány v systému DarkFactory?*

Práce vychází z předpokladu, že jazykový model je v současných agentických systémech pouze jednou částí širšího celku. Praktické použití coding agentů proto závisí také na způsobu práce s kontextem, nástroji a trvalým stavem, na orchestraci jednotlivých kroků, automatickém ověřování a zapojení člověka do rozhodovacích bodů. Teoretická část tyto opakující se principy popisuje a praktická část ukazuje jejich propojení v jednom konkrétním systému.

Praktická část má podobu inženýrské případové studie DarkFactory, agentické softwarové továrny nativně postavené na GitHubu. Systém byl implementován pomocí komerčních coding agentů a je záměrně navržen jako jednoduchá počáteční implementace. Již v této podobě dokáže sama spouštět a řídit coding agenty a vytváří tak základ, který lze pomocí stejného procesu dále rozvíjet. Cílem případové studie není měřit obecnou úspěšnost jazykových modelů ani porovnávat jednotlivé coding agenty, ale popsat konkrétní realizaci současných postupů agentického inženýrství a ukázat, co jejich spojení umožňuje v praxi.

1.3 Terminologie

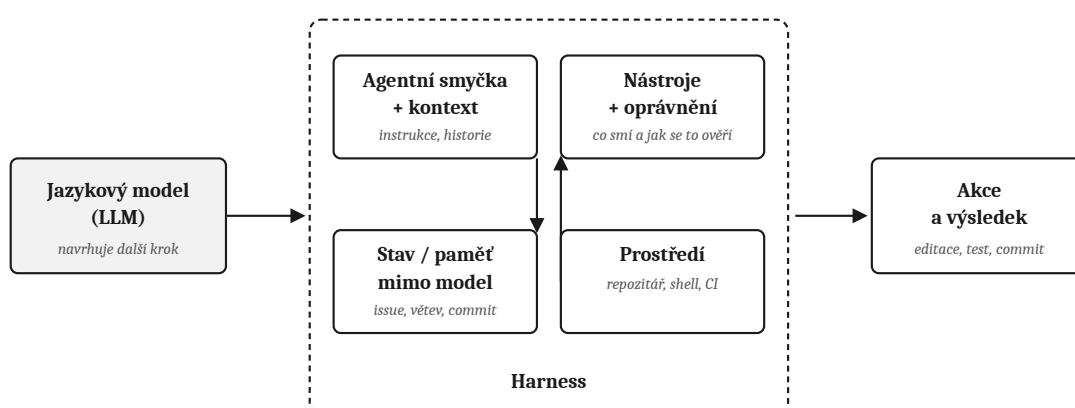
Překlad anglických výrazů v této práci sleduje jedno pravidlo. Co anglicky nazýváme **agent**, je česky **agentní** — agent, agentní krok, agentní smyčka. Co anglicky nazýváme **agentic**, je česky **agentické** — agentický systém, agentické inženýrství. Podstatné jméno **agent** se tedy nepřekládá, protože označuje samotného jednajícího, zatímco přípona **-ic** znamená schopnost jednat, nikoli samotného jednajícího (9). Podle téhož pravidla je *Agent Loop* česky *agentní smyčka*, protože jde o smyčku agenta, a nikoli o vlastnost systému.

2 Teoretická část

2.1 Agent: Co to je a jak funguje

Agent je model plus harness (10). Model navrhuje a harness mu dopředuje kontext, nástroje a pravidla, jejichž prostřednictvím se návrh mění v čin. Celý systém tvoří několik vrstev — rozhraní, smyčka agenta, oprávnění, nástroje, uložený stav a prostředí, v němž běží — a model je jen jedna z nich. Spolehlivost agenta proto neurčuje samotný model, ale skladba těch vrstev. Čím jsou modely schopnější, tím menší část práce zbývá pro tuto skladbu a tím víc práce přebírá model. Rozvrstvení shrnuje Obrázek 4.

Hranice, za kterou model přestává být celou soustavou



Model je uvnitř harnessu, ne nad ním. Repořitář, oprávnění ani trvalý stav mu nedává samotný; dodává je vrstva, která jeho návrh převádí na akci, jejíž výsledek lze dohledat.

Obrázek 4: Od jazykového modelu k agentickému systému. Jazykový model je jednou ze čtyř součástí harnessu; repořitář, oprávnění ani trvalý stav mu nedává samotný, ale vrstva, která jeho návrh převádí na akci s dohledatelným výsledkem.

2.1.1 Jazykový model (LLM)

Současný jazykový model stojí na architektuře **Transformer**, kterou představil Google v roce 2017 v práci „**Attention Is All You Need**“ (11).

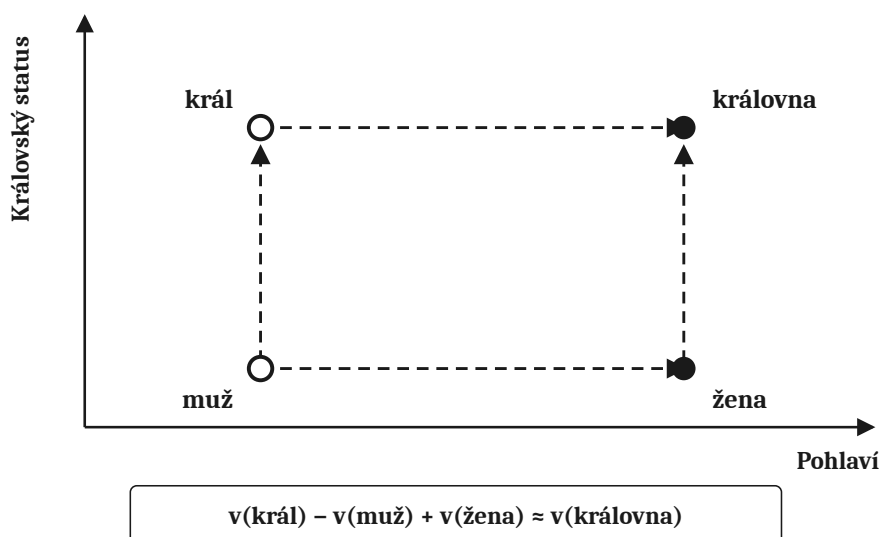
Její základem je **attention**: ke každému tokenu se připočítá vážený součet hodnot všech ostatních, takže jeho reprezentace nese informaci z celého kontextu a vzdálenost mezi pozicemi není pevně daná jejich pořadím. Všechny tokeny se vyhodnocují proti sobě najednou, ale náklady rostou s druhou mocninou délky kontextu.

Jazykový model je sám o sobě jen funkce. Na základě toho, co dostane v kontextu, vypočítá rozložení pravděpodobností nad následujícím tokenem a jeden z nich vybere. Mezi voláními si nic nepamatuje, nezná nic mimo text, který mu byl předložen, a nemá přístup k souborům,

příkazům ani stavu práce. Všechno, co o zadání ví, je v kontextu, a jinudy se to do něj nedostane.

Při **inferenci** tedy model zpracuje obsah kontextového okna a vytváří výstupní tokeny jeden za druhým, přičemž každý nový token se stane součástí kontextu pro další krok. To je celá jeho schopnost: převést kontext na posloupnost tokenů. Stav mezi kroky, nástroje, ukládání výsledků a oprávnění musí přijít zvenčí — a to je právě to, co doplňuje harness.

Vektorové reprezentace, označované jako **embeddingy**, zachycují sémantické vztahy v prostoru vektorů. Známým příkladem je vztah mezi vektory slov král, královna, muž a žena (12) Obrázek 5.



Obrázek 5: Ilustrace sémantického vztahu mezi vektorovými reprezentacemi slov *král*, *královna*, *muž* a *žena* (12).

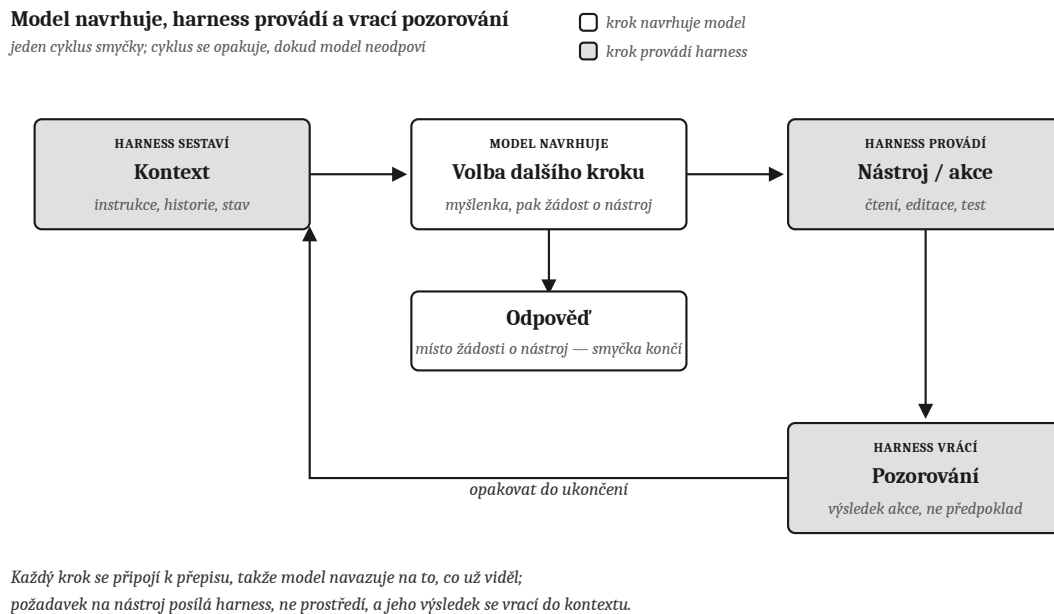
2.1.2 Smyčka (Loop)

Základním mechanismem agentického systému je **agentní smyčka**: konkrétní implementace vzoru **ReAct** (*Reasoning and Acting*) (13) v daném harnessu. Jeden cyklus má pět kroků:

- sestavit kontext: systémové pokyny, dosavadní přepis, stav prostředí;
- nechat model odpovědět;
- je-li v odpovědi požadavek na nástroj, provést jej;
- zapsat výsledek nástroje jako pozorování;

- vrátit se na první krok.

Model přitom v každém cyklu nejprve zdůvodní, co udělá, a teprve pak žádá o nástroj (13). Harness akci provede mimo model a výsledek mu vrátí; ten se připojí k přepisu a cyklus běží dál. Smyčka končí teprve tehdy, když model místo požadavku na nástroj vydá závěrečnou odpověď. V prosté posloupnosti promptů by model nemohl poznat, zda předchozí krok uspěl; průběh shrnuje Obrázek 6.



Obrázek 6: Vzor ReAct: myšlenku navrhuje model, akci provádí harness a pozorování mu vrátí. Každý krok se připojí do kontextu, takže smyčka pokračuje na tom, co už model viděl. Končí odpovědí místo dalšího požadavku na nástroj (13).

2.1.3 Nástroje (Tools)

Zadání určuje, co má agent udělat; co mu je dovoleno udělat, určuje harness. Jde o dvě roviny, které se dají měnit nezávisle, a odpovídají běžnému výčtu součástí harnessu (10):

- **Nástroje** umožňují agentovi číst a upravovat soubory, vyhledávat nebo spouštět příkazy (14).
- **Skilly** spojují opakovaně použitelné instrukce, skripty a zdroje pro určitý typ úlohy (15).
- **Hooks** reagují na události životního cyklu a mohou před akcí či po ní vynutit kontrolu (16).
- **Model Context Protocol (MCP)** standardizuje napojení externích nástrojů a datových zdrojů prostřednictvím rozhraní klient–server (17).

Každá akce ještě předtím projde oprávněním, které pro ni platí: provést automaticky, zeptat se člověka, nebo zakázat. Ani když model navrhne něco nebezpečného, dostane se to k provedení jen tehdy, když to oprávnění dovolí.

Instrukce pro coding agenty lze verzovat přímo s repozitářem. Otevřený standard `AGENTS.md`, podporovaný řadou současných coding agentů, poskytuje předvídatelné místo pro projektové instrukce, například příkazy sestavení, testy a konvence (18). Pro rozšiřitelné schopnosti se používá standard **Agent Skills**, v němž je každý skill adresář obsahující soubor `SKILL.md` (15). Konkrétní umístění těchto adresářů se mezi harnessy liší; některé systémy používají `.agents/skills/`, jiné vlastní adresáře.

2.1.4 Kontext (Context)

Kontextové okno zahrnuje pracovní kontext jednoho volání modelu: instrukce, části repozitáře, historii volání nástrojů i výsledky předchozích kroků. Jeho velikost sama o sobě nezaručuje, že model všechny podstatné informace správně využije. Liu et al. ukázali, že výkon modelů při práci s dlouhým kontextem závisí mimo jiné na pozici relevantní informace a může výrazně klesat, pokud se důležitá informace nachází uprostřed vstupu (19).

S rostoucím množstvím kontextu se může obecně zhoršovat schopnost modelu rozlišovat a využívat podstatné informace; tento problém se označuje jako **context rot** (20). Jedním ze způsobů, jak u dlouhých agentních běhů udržet kontext v použitelném rozsahu, je **kompakce** (*compaction*), při níž jsou starší části průběhu nahrazeny kratším souhrnem důležitých rozhodnutí, výsledků a otevřených úkolů (20).

Od tohoto omezení je potřeba odlišit kvalitu samotného obsahu kontextu. Zastaralé instrukce, rozpory mezi soubory nebo již neplatný stav mohou agenta vést špatným směrem bez ohledu na velikost kontextového okna. Proto je vhodné uchovávat autoritativní dlouhodobý stav mimo samotný přepis konverzace a do kontextu v každém kroku vybírat aktuální informace, které agent skutečně potřebuje.

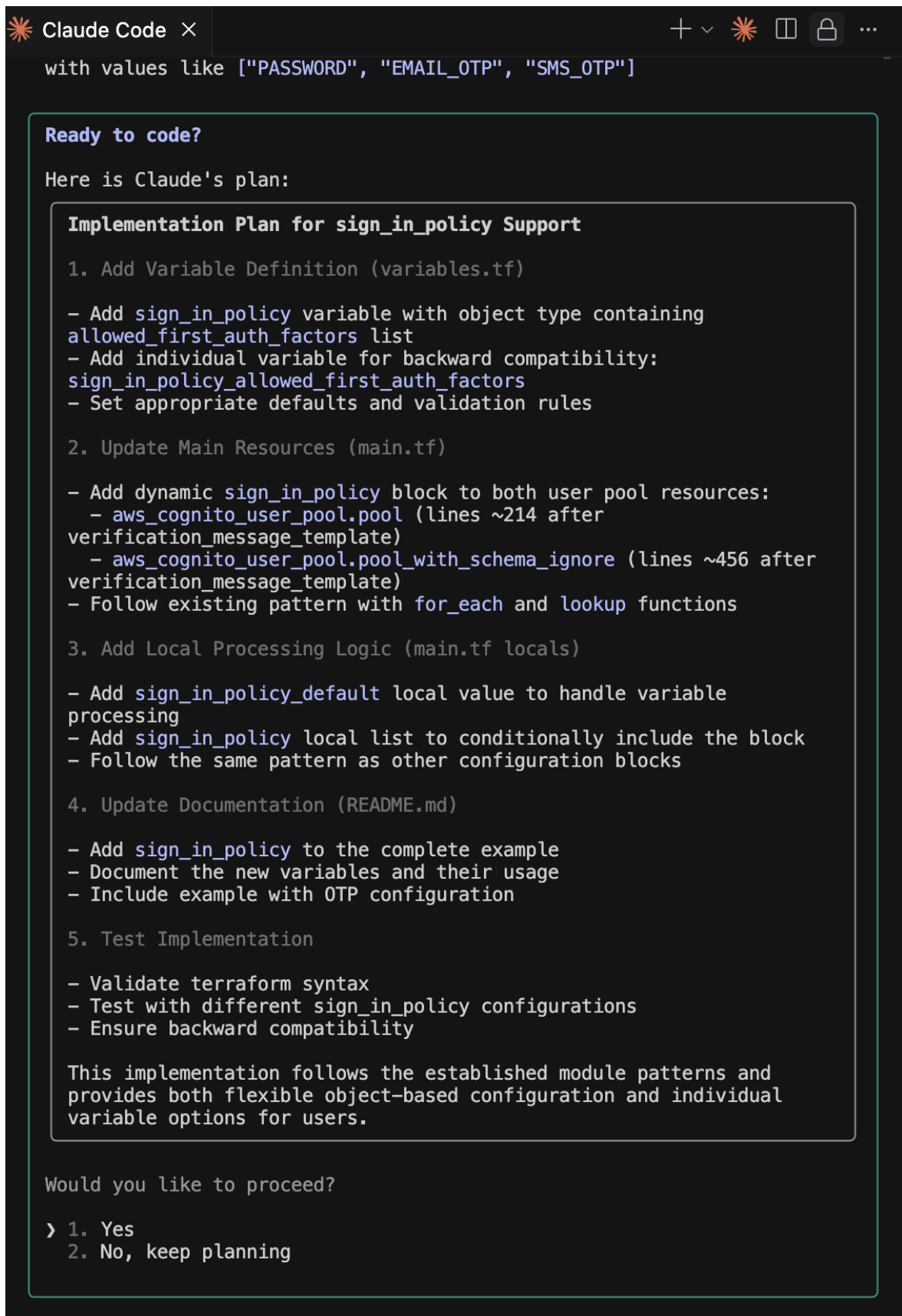
2.2 Agentické inženýrství

V této práci se **agentickým inženýrstvím (Agentic Engineering)** rozumí soubor postupů pro návrh a řízení vývoje softwaru pomocí coding agentů. Jeho předmětem není samotný model, ale širší systém, v němž agent pracuje: zadání, kontext, nástroje, omezení, orchestrace, pravidla integrace a odpovědnost člověka. Tomu odpovídá i popis role **agentického inženýra**, jehož práce se vedle samotného kódu přesouvá k formulaci zadání, řízení agentních běhů a kritickému posouzení strojem vytvořených výstupů (21). Mezi hlavní oblasti patří **Prompt engineering** (22), **Context engineering** (20), **Harness engineering** (23; 24), **Loop Engineering** (25) a **Workflow/Graph Engineering** (26). Tyto postupy umožňují delegovat ohraničenou část vývojové práce agentovi, zatímco záměr, hranice procesu a integrace výsledku zůstávají explicitně řízené.

2.2.1 Specifikace (Specification)

Spec-first development staví specifikaci před samotnou implementaci. Tento princip nevznikl s coding agenty; navazuje na dlouhodobou praxi **requirements engineering**, v níž jsou požadavky a očekávané vlastnosti systému popsány před nebo v průběhu jeho realizace. IEEE vydalo samostatný standard pro *Software Requirements Specification* už v roce 1984 (27) a současný standard ISO/IEC/IEEE 29148 formalizuje procesy a výstupy requirements engineering v průběhu životního cyklu softwaru (28).

U coding agentů získává tento přístup nový význam. Specifikace už neslouží pouze jako podklad pro lidského vývojáře, ale může být přímo předána agentovi jako vymezení cíle, požadavků a omezení implementace. Čím samostatněji má agent pracovat, tím důležitější je, aby bylo před zahájením implementace dostatečně přesně určeno, **co má být výsledkem**, zatímco konkrétní postup může agent zvolit sám. Příklad oddělení návrhu plánu od jeho provedení ukazuje Obrázek 7.



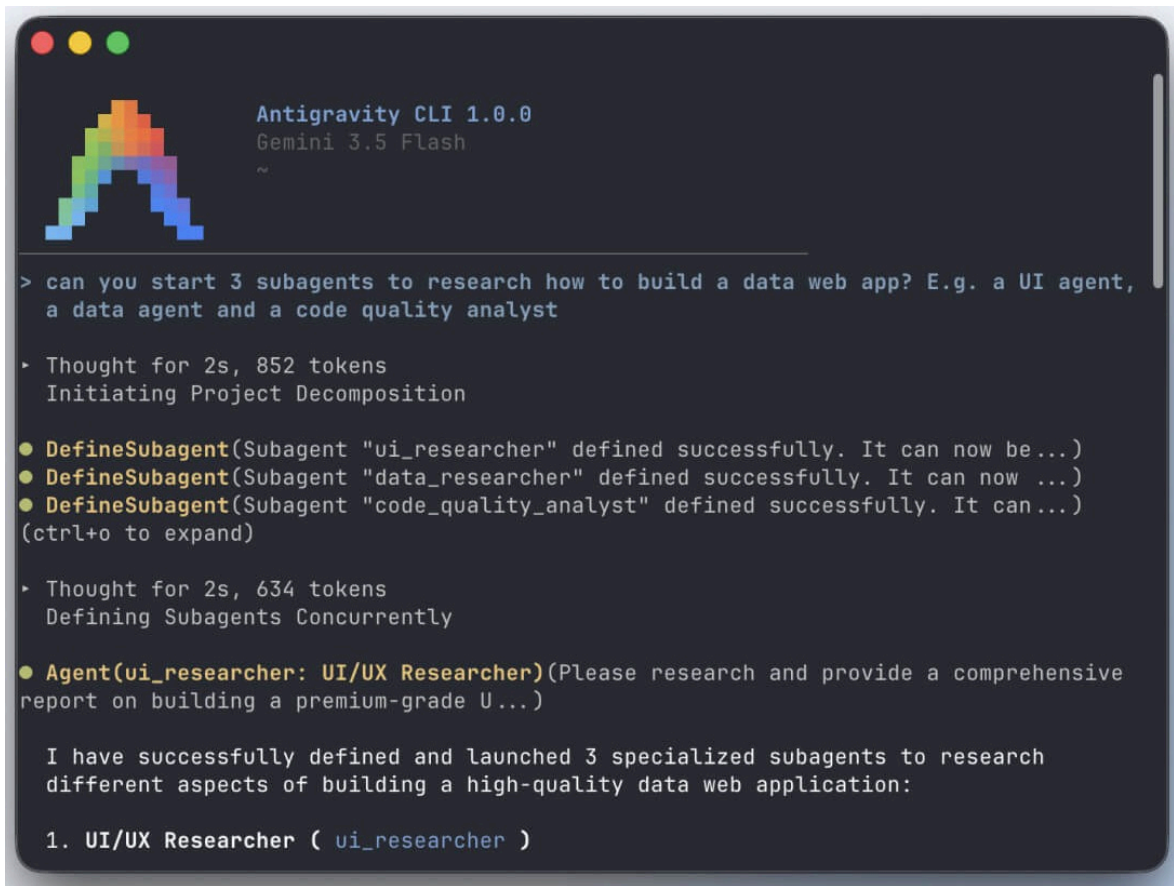
Obrázek 7: Plán, který model navrhne, než se začne psát kód: seznam kroků podle souborů, které se mění, a otázka, zda se má pokračovat. Znění plánu je tak oddělené od jeho provádění — je to rozhodnutí, které člověk schvaluje nebo odmítne, nikoli popis práce, která už běží. Snímek z Claude Code, převzato z (29).

Prompt engineering se soustředí na formulaci instrukcí, omezení, příkladů a očekávaného výstupu konkrétního inferenčního kroku (22). **Context engineering** řeší širší a průběžný výběr, uspořádání, obnovování a kompakci informací, které má model v daném kroku k dispozici (20).

2.2.2 Orchestrace (Orchestration)

Více agentů samo o sobě nezaručuje lepší výsledek. Paralelizace přináší užitek jen tehdy, když jsou omezeny vzájemné závislosti a koordinátor dokáže odhalit konflikty, ověřit dílčí výstupy a posoudit sloučený výsledek vůči společným podmínkám přijetí. Orchestrace proto zahrnuje nejen rozdělení práce, ale také správu kontextu, pořadí kroků, sdílený stav a integrační kontroly. Je zvláště důležitá u rozsáhlých úloh, které se nevejdou do jednoho kontextového okna. Samotná kompakce sice umožňuje v práci pokračovat, ale nemusí zachovat všechny detaily potřebné pro jednotlivé části úlohy. Rozdělení práce mezi více běhů nebo agentů umožňuje každé části pracovat s vlastním relevantním kontextem a jejich výsledky následně koordinovaně spojit. Čtyři vzorce na to odpovídají:

- **Coordinator/subagent** rozdělí rozsáhlou úlohu na dílčí běhy. Koordinátor deleguje podúkoly specializovanému subagentovi s vlastním kontextem a přebírá jeho výsledek, nezávislé podúkoly zpracují paralelní pracovníci Obrázek 8.



```
Antigravity CLI 1.0.0
Gemini 3.5 Flash

> can you start 3 subagents to research how to build a data web app? E.g. a UI agent,
a data agent and a code quality analyst

▸ Thought for 2s, 852 tokens
Initiating Project Decomposition

● DefineSubagent(Subagent "ui_researcher" defined successfully. It can now be...)
● DefineSubagent(Subagent "data_researcher" defined successfully. It can now ...)
● DefineSubagent(Subagent "code_quality_analyst" defined successfully. It can...)
(ctrl+o to expand)

▸ Thought for 2s, 634 tokens
Defining Subagents Concurrently

● Agent(ui_researcher: UI/UX Researcher)(Please research and provide a comprehensive
report on building a premium-grade U...)

I have successfully defined and launched 3 specialized subagents to research
different aspects of building a high-quality data web application:

1. UI/UX Researcher ( ui_researcher )
```

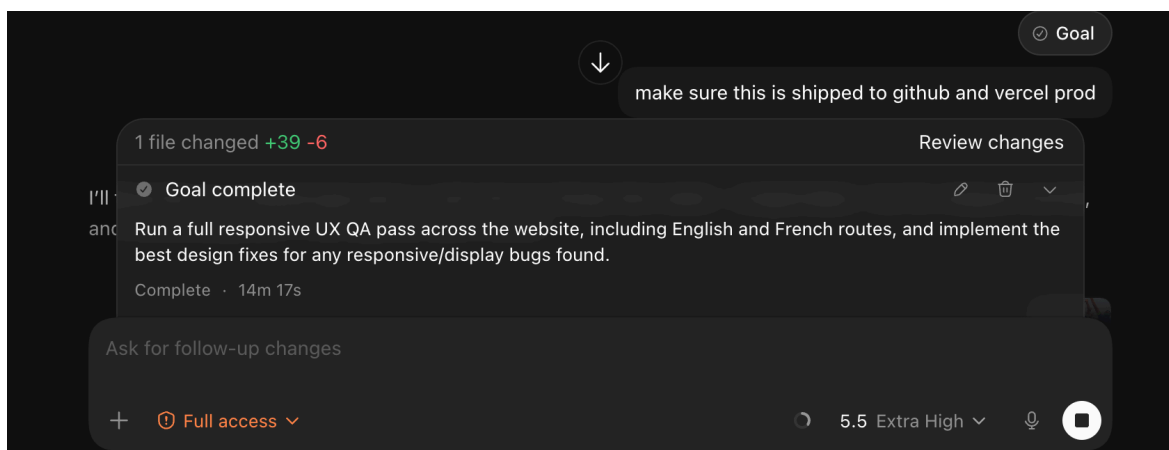
Obrázek 8: Rozhraní Google Antigravity CLI, tedy harnessu agenta běžícího v terminálu: koordinátor definuje tři specializované subagenty a spouští je souběžně; každý běží vlastním kontextem (30).

- **Workflow graph** předem určuje závislosti, pořadí a větvení fází (26). Claude Code u dynamických workflow napíše skript, který runtime vykonává na pozadí, takže smyčku, větvení i mezivýsledky drží skript místo kontextu modelu a plán je program, který lze přechíst a znovu spustit (31) Obrázek 9.

Phases	CodeReview · 16 agents
✓ Foundation 8/8	✓ cloud:security claude-codex-pro 47.2k tok · 15 tools · 1m 32s
✓ AcceptanceCriteria 4/4	✓ cloud:database claude-deepseek-pro 141.9k tok · 59 tools · 7m 14s
✓ CodeReview 16/16	✓ cloud:python Sonnet 4.6 120.4k tok · 39 tools · 12m 38s
✓ Verify 39/39	✓ cloud:stripe-deep claude-deepseek-pro 133k tok · 46 tools · 5m 5s
5 Completeness 3/4	✓ cloud:signup-deep claude-deepseek-pro 127.2k tok · 46 tools · 5m 58s
6 Synthesis	✓ gubbi:security claude-codex-pro 34.9k tok · 17 tools · 1m 14s
	✓ gubbi:database claude-deepseek-pro 117.9k tok · 73 tools · 7m 36s
	✓ gubbi:mcp-deep claude-deepseek-pro 113.1k tok · 48 tools · 7m 12s
	✓ gubbi:extraction-deep (claude-deepseek-pro 131.4k tok · 97 tools · 10m 18s
	✓ gubbi:python Sonnet 4.6 73.1k tok · 27 tools · 5m 19s
	✓ common:deep claude-deepseek-pro 134.7k tok · 56 tools · 5m 45s
	✓ common:security claude-codex-pro 21.5k tok · 9 tools · 1m 16s
	✓ testbench:deep claude-deepseek-pro 103.8k tok · 79 tools · 6m 16s
	✓ infra:devops claude-codex-pro 67.9k tok · 33 tools · 8m 46s
	✓ infra:security claude-codex-pro 112.4k tok · 40 tools · 5m 51s
	✓ infra:architecture claude-deepseek-pro 94k tok · 42 tools · 5m 57s

Obrázek 9: Postup práce dynamického workflow v Claude Code: vlevo fáze plánu se stavem, vpravo rozvinutá fáze *CodeReview* se 16 agenty, u každého model, spotřebované tokeny, počet nástrojů a doba běhu. Snímek pochází z komunitního příspěvku; rozhraní i jeho chování popisuje dokumentace (31).

- **Goal loop** je nadřazená řídicí smyčka. Po dílčím dokončení ReAct smyčky se pozorovaný stav porovná s cílem a podmínkami přijetí a běh skončí, nebo pokračuje další iterací. Vzor pochází z tzv. **Ralph loop**, který v roce 2025 popsal Geoffrey Huntley jako smyčku, jež opakovaně spouští agenta se stejným zadáním a stav si drží v pracovním stromu, nikoli v přepis konverzace (32). Dnes je to běžná funkce Claude Code i Codexu (25; 33) Obrázek 10.



Obrázek 10: Cíl, proti němuž agent postupuje, a jeho splnění: po dokončení dílčího kroku se stav porovná s cílem a běh v tomto případě končí, včetně doby, kterou zabral. Snímek pochází z komunitního příspěvku; funkci popisuje dokumentace (25).

- **human-in-the-loop (HITL)** označuje bod, v němž se automatizovaná smyčka doplní o kontrolní brány vyžadující explicitní lidské rozhodnutí, například schválení specifikace, potvrzení implementačního plánu nebo přijetí výsledného diffu. Vývojář tak může pone-

chat agentovi ohraničenou implementační práci a současně si zachovat odpovědnost za záměr a integraci.

2.3 Softwareová továrna

Softwareová továrna označuje prostředí, které převádí vývoj softwaru do co nejvíce řízeného a opakovatelného procesu. Samotná myšlenka není nová. Robert W. Bemer ji už na konferenci NATO o softwarovém inženýrství v roce 1968 popsal jako *machine-controlled production environment, or software factory* (34). V jeho návrhu měly konstrukce programu, jeho kontrola i používání probíhat uvnitř společného prostředí vybaveného potřebnými nástroji pro práci se soubory, kompilaci, testování a sestavení systému.

Tradiční automatizace se uplatňuje především tam, kde lze postup předem přesně popsat programem. Coding agenti tuto hranici posouvají: jazykový model může podle aktuálního stavu systému volit další kroky a používat nástroje i v úlohách, jejichž přesný postup není předem pevně určen. Softwareová továrna tak může kombinovat běžnou deterministickou automatizaci s agentními kroky a lidským rozhodováním.

Takové uspořádání se již používá v reálném softwarovém inženýrství. Společnost Stripe o svých interních agentech *Minions* uvádí: „Minions now ship over 1,000 PRs to production every week. In all these cases, humans just have to review and approve.“ (7) Meta podobně popisuje vlastní agentní platformu, která automatizuje hledání a opravu výkonnostních problémů a dokáže dovést proces až k pull requestu připravenému k lidské revizi (8).

DarkFactory, popsaná v praktické části této práce, představuje záměrně jednoduchý **bootstrap** tohoto přístupu. Její počáteční implementace už dokáže coding agenty spouštět a řídit v rámci repozitářového workflow a zároveň vytváří základ, který lze pomocí stejné pipeline dále rozvíjet.

PRAKTICKÁ ČÁST

DarkFactory

3 Praktická část

3.1 Metodika

Praktická část má podobu inženýrské případové studie systému DarkFactory, konkrétní GitHub-native realizace širšího vzoru softwarové továrny popsaného v teoretické části. Cílem není měřit obecnou úspěšnost jazykových modelů ani porovnávat konkrétní coding agenty, ale na jednom reálném systému ukázat, jak se principy popsané v teoretické části propojují v produkčním vývojovém procesu.

Popis implementace a následná zjištění se vztahují ke konkrétní revizi DarkFactory `d576ec8f` (35). Tím je případová studie oddělena od pozdějšího vývoje systému a jednotlivá tvrzení lze vztáhnout ke stejnému stavu repozitáře. Analýza vychází ze zdrojového kódu, GitHub Actions workflow, agentního runneru, registru harnessů, projektových pravidel a dokumentační konfigurace této revize.

Postup případové studie navazuje na teoretickou část. Nejprve jsou v současných zdrojích vymezeny opakující se principy agentického vývoje, zejména práce s kontextem a trvalým stavem, specifikace, orchestrace, rozdělení odpovědností mezi model a harness, programové kontroly a lidské rozhodovací body. Následně je u každého z nich sledováno, jak je realizován v DarkFactory, a z implementace je rekonstruován celý průchod požadavku systémem. Diskuse potom porovnává tento konkrétní případ s širším vzorem a odděleně uvádí omezení, která vyplývají ze skutečného chování zkoumané revize.

Součástí případové studie je také návrh a implementace DarkFactory pomocí komerčních coding agentů. Systém je záměrně jednoduchý a jeho počáteční podoba slouží jako bootstrap: již dokáže sama spouštět a řídit coding agenty a vytváří tak základ, který lze pomocí stejného procesu dále rozvíjet. Praktická část proto vedle výsledné architektury sleduje i způsob, jakým se v jednom vývojovém procesu doplňují coding agent, harness, programová orchestrace a člověk.

3.2 Návrh systému DarkFactory

DarkFactory byla navržena jako záměrně malá, GitHub-native realizace principů popsaných v teoretické části. Cílem první verze nebylo pokrýt co nejvíce funkcí, ale uzavřít nejmenší prakticky použitelný vývojový cyklus: přijmout požadavek, předat otevřenou inženýrskou práci coding agentovi, zachovat průběžný stav a vrátit výsledek do běžného GitHub workflow. Počáteční rozsah byl zároveň zvolen tak, aby bylo možné první funkční podobu vytvořit v jednom souvislém agentním běhu a následné rozšiřování už provádět pomocí stejné pipeline.

Základním návrhovým rozhodnutím je použití GitHubu nejen jako hostingu repozitáře, ale také jako hlavního rozhraní a trvalé stavové vrstvy systému. Požadavek, diskuse, schválení, větev, commity a pull request proto zůstávají mimo kontext modelu a přežívají jednotlivé agentní běhy. Události GitHubu spouštějí příslušná GitHub Actions workflow, takže DarkFactory pro základní průchod nepotřebuje vlastní trvale běžící server. Každý agentní krok probíhá odděleně a pracovní prostředí je izolováno kontejnerem (35).

DarkFactory záměrně neimplementuje vlastní coding-agent runtime. Produkční harness zajišťuje agentní smyčku, komunikaci s modelem, nástroje a práci s jeho kontextem; DarkFactory nad tím řídí pořadí kroků, předávání vstupů a výstupů, stav procesu a lidské brány. Otevřené úlohy, jako interpretace požadavku, plánování, implementace nebo revize, řeší model prostřednictvím harnessu. Kroky, které lze určit přesněji, například spuštění workflow, práce s větví, testy, formátování nebo změna stavu, zůstávají programově řízené (35).

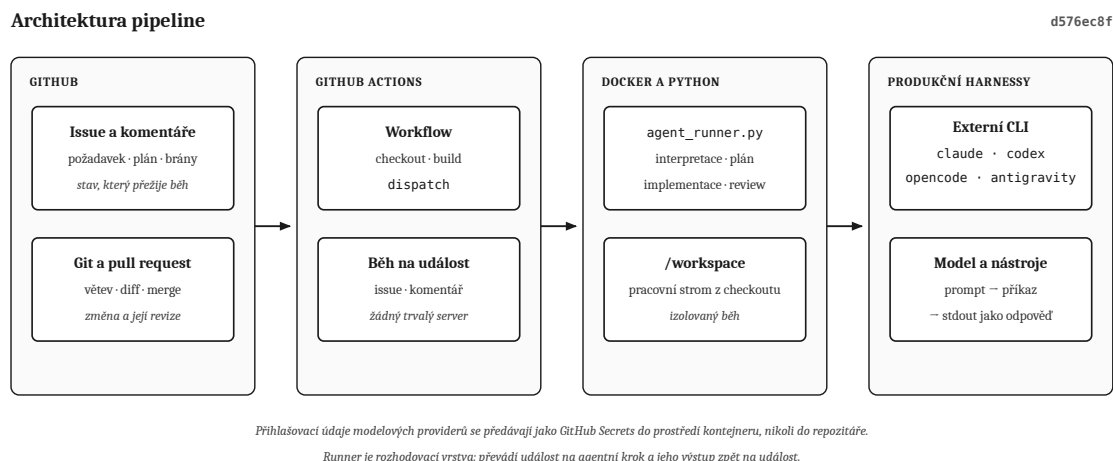
Důležitou součástí návrhu je dokumentace jako rozhraní mezi strojově prováděným procesem a člověkem. Ve zkoumané revizi vyžadují pravidla projektu úplnou dokumentaci veřejného API přímo ve zdrojovém kódu a současně generování dokumentačního webu z kanonických souborů repozitáře. Dokumentace se tedy neudrží jako druhá ručně psaná kopie systému; publikační vrstva vzniká ze stejných zdrojů, podle kterých pracují agenti a automatizace. Tím dokumentace propojuje konkrétní implementaci s člověkem, který potřebuje systém pochopit, kontrolovat a dále směřovat (35).

Návrh zachovává explicitní rozhodovací body člověka. Člověk nemusí řídit jednotlivé technické kroky, ale schvaluje záměr a plán ještě před implementací a na konci rozhoduje o přijetí výsledku. Mezi těmito body pokračuje workflow automaticky. Rozdělení odpovědností je proto záměrné: model provádí otevřený inženýrský úsudek, harness poskytuje agentní schopnosti, DarkFactory řídí proces a stav a člověk určuje, zda se proces může posunout přes klíčové hranice (35).

Tato první verze současně tvoří bootstrap pro další vývoj DarkFactory. Systém byl sám vytvořen pomocí komerčních coding agentů a po uzavření základního cyklu už dokáže stejné nástroje spouštět pro práci na vlastním repozitáři. Smyslem jednoduchého začátku tedy není zůstat u malé implementace, ale co nejrychleji získat funkční pipeline, kterou lze následně po menších krocích používat k rozšiřování sebe sama. Následující část proto ukazuje, jak jsou tato návrhová rozhodnutí realizována konkrétními komponentami systému.

3.3 Architektura systému

Návrhové principy z předchozí části jsou v praktické implementaci rozděleny mezi GitHub, GitHub Actions, izolované pracovní prostředí a produkční coding-agent harness. GitHub uchovává vývojový stav a události, Actions spouští jednotlivé fáze, Docker odděluje agentní běh a pythonovský runner převádí stav workflow na konkrétní agentní krok. Samotnou agentní smyčku, práci s modelem a nástroji zajišťuje zvolený harness (35). Toto rozdělení znázorňuje Obrázek 11.



Obrázek 11: Architektura DarkFactory: GitHub poskytuje události a trvalý vývojový stav, GitHub Actions výpočetní prostředí, Docker izolaci běhu a pythonovský runner převádí událost na agentní krok. Produkční harness zajišťuje model, nástroje a agentní smyčku (35).

GitHub v tomto uspořádání neplní pouze roli vzdáleného repozitáře. Issue uchovává uživatelský požadavek, interpretaci a navazující diskusi, pracovní větev a commity nesou samotnou implementaci a pull request slouží jako místo automatické i lidské revize. Stav tak není závislý na jednom kontextovém okně modelu a jednotlivé agentní běhy mohou na předchozí práci navazovat přes objekty, které jsou součástí běžného vývojového workflow (35).

Události nad těmito objekty spouští GitHub Actions. Workflow `agent.yml` reaguje na otevření issue, nový komentář, review komentář, `repository_dispatch` nebo ruční spuštění. V každém běhu se připraví cílový repozitář, sestaví se agentní obraz podle `docker/Dockerfile.agent` a runner je spuštěn uvnitř kontejneru nad checkoutnutým pracovním stromem. DarkFactory tím používá GitHub Actions současně jako událostní mechanismus i jako krátkodobé výpočetní prostředí, aniž by základní pipeline vyžadovala vlastní trvale běžící službu (35).

Pythonovský `agent_runner.py` je orchestrace nad harnessem, nikoli jeho náhrada. Rozlišuje jednotlivé fáze, například interpretaci, plánování, implementaci, automatickou revizi, kontrolu souladu s plánem a reakci na zpětnou vazbu. Současně zpracovává stav procesu a výstupy jednotlivých kroků. Vlastní práce coding agenta je oddělena prostřednictvím registru v `harnesses.py`: každý podporovaný harness deklaruje binární soubor, způsob sestavení příkazové řádky, autentizaci a modelové fallbacky. Pořadí harnessů lze měnit konfigurací, takže orchestrace nemusí znát konkrétní CLI, které modelový krok právě provádí (35).

Odděleně jsou řešeny také přihlašovací údaje. Operace nad GitHubem používají autorizovaný token nebo GitHub App, zatímco přihlašovací údaje poskytovatelů modelů vstupují do workflow jako GitHub Secrets a jsou předány pouze běhu, který je potřebuje. Nejsou součástí issue, commitů ani jiného trvalého záznamu práce (35).

Součástí architektury je i dokumentační vrstva. Ve zkoumané revizi pravidla vyžadují dokumentaci veřejného API přímo ve zdrojových souborech a dokumentační web se generuje z těchto inline popisů a z kanonických repozitářových dokumentů. Publikovaná dokumentace tak není samostatnou ručně udržovanou kopií systému, ale čitelným pohledem nad jeho zdroji. Pro člověka představuje most mezi automatizovaným procesem a konkrétním kódem, ze kterého tento proces vzniká (35).

Samotný seznam komponent ještě nevysvětluje, jak spolupracují při skutečném vývojovém požadavku. Následující část proto sleduje celý průchod od zadání až po integraci výsledku.

3.4 Průchod požadavku systémem

Praktický průchod začíná vytvořením issue s uživatelským požadavkem. Repozitář pro něj nabízí předlohu se šesti poli, z nichž tři jsou povinná. Vedle doslovného znění požadavku se uvádí oblast a typ změny; další kontext nebo navržená kritéria přijetí jsou volitelná (35). Strukturu předlohy shrnuje Tabulka 1.

Pole předlohy issue

Verbatim User Request povinné

Area / Component povinné

Request Type povinné

Parent Epic volitelné

Proposed Acceptance Criteria volitelné

Additional Context volitelné

Tabulka 1: Pole předlohy issue pro uživatelský požadavek: tři jsou povinná a tři volitelná (35).

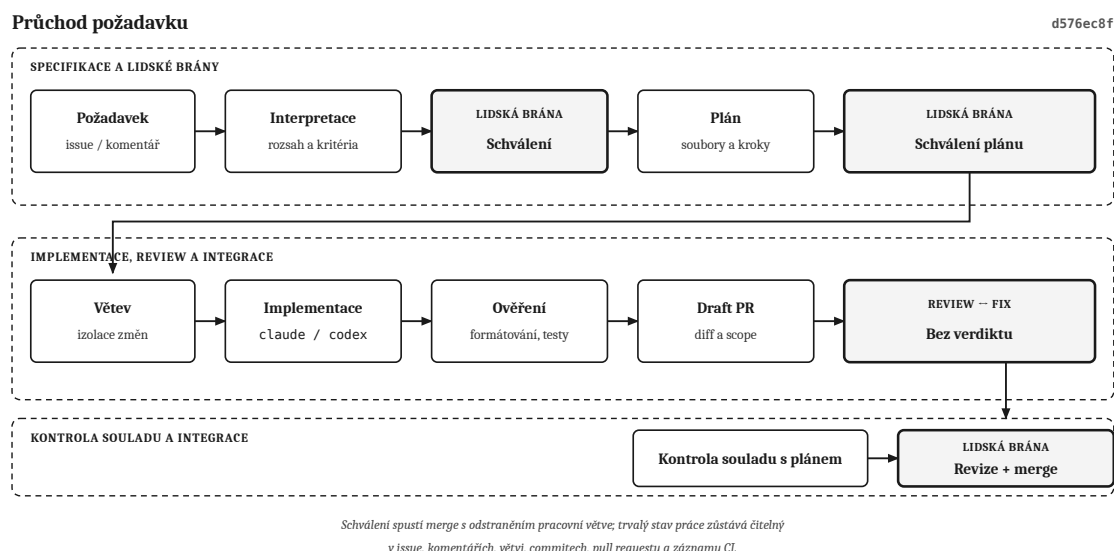
Runner načte požadavek a předá jej modelu k interpretaci. Výsledek zapíše jako komentář k issue a další fáze čeká na lidské schválení. Tím se odděluje otázka, zda agent zadání pochopil, od otázky, jak jej bude implementovat. Teprve po schválení interpretace runner vytvoří propojené child issue typu `Plan:`, ve kterém agent sestaví implementační plán. Plán vymezuje očekávané změny, dotčené části repozitáře a způsob ověření. Také tento krok vyžaduje lidské schválení před vznikem pracovní větve (35).

Po schválení plánu runner vytvoří nebo obnoví pracovní větev a předá coding agentovi schválený plán spolu s omezením jeho rozsahu. Harness pracuje přímo nad checkoutnutým repozitářem, upravuje soubory a používá nástroje dostupné uvnitř kontejneru. Po implementaci runner spustí dostupné formátovací a testovací příkazy. Jejich výstup je součástí dalšího agentního kroku `fix`, takže zjištěný problém může být opraven ještě před další revizí. Poté je změna uložena do commitu, větev odeslána a otevřen draft pull request (35).

Na draft pull requestu pokračuje automatická review smyčka. Model obdrží aktuální diff a stav větve, hledá konkrétní problémy a případný nález vrací do opravného běhu. Současně se nad větví znovu spouští CI. Smyčka je omezená na tři iterace, aby agent nemohl spotřebovávat prostředky bez konce. Pokud ani poslední iterace neskončí čistým verdiktem, runner přesto pokračuje k následující kontrole souladu s plánem (35). Pokud revize vyžaduje změnu mimo schválený plán, runner ji zapíše jako `Plan Deviation` s odůvodněním místo toho, aby ji skryl uvnitř implementace.

Po automatické revizi následuje samostatná kontrola souladu výsledného diffu se schváleným plánem. Jde o jiný modelový dotaz než běžná code review: neposuzuje obecnou

kvalitu změny, ale zda implementace odpovídá tomu, co člověk předem schválil. Teprve po úspěšném dokončení této fáze může být výsledek předán k lidské revizi (35). Celý sled fází shrnuje Obrázek 12.



Obrázek 12: Průchod požadavku DarkFactory: interpretace a plánování jsou odděleny lidskými bránami, implementace probíhá na pracovní větvi a draft pull request prochází automatickou revizí a kontrolou souladu s plánem před lidským přijetím výsledku (35).

Člověk může na pull requestu změnu schválit nebo vrátit zpětnou vazbu. Změnový požadavek spustí opravný běh na stejné větvi; agent dostane původní plán, konkrétní připomínku a aktuální stav implementace, provede úpravu a vrátí výsledek znovu do automatické revize. Není tedy nutné zakládat nový požadavek pokaždé, když lidská revize odhalí problém (35).

Konečné schválení zpracovává samostatné workflow. Po ověření autorizovaného aktéra a splnění požadovaných podmínek převede pull request do stavu připraveného ke sloučení a provede merge s odstraněním pracovní větve. Celý průchod tak střídá tři druhy kroků: otevřený inženýrský úsudek modelu, deterministickou automatizaci a explicitní lidská rozhodnutí v bodech, kde se mění záměr nebo přijímá výsledek (35).

3.5 Implementace pomocí coding agentů

DarkFactory není pouze systém určený pro práci coding agentů; coding agenti byli použiti i při jeho samotném vývoji. Člověk určoval záměr, omezení a zásadní rozhodnutí, zatímco otevřená implementační práce byla delegována komerčním coding agentům. Praktická část tak nepopisuje agentické inženýrství pouze zvenčí, ale používá stejné postupy i při vzniku zkoumaného artefaktu.

Počáteční rozsah byl záměrně omezen na nejmenší uzavřený proces, který dokáže přijmout požadavek, předat inženýrskou práci agentovi a vrátit výsledek do řízeného GitHub workflow. Cílem nebylo předem ručně vytvořit rozsáhlou finální platformu, ale dostat první funkční podobu do rozsahu jednoho souvislého agentního běhu. Tím se zkracuje cesta k bodu, ve kterém už systém může být použit pro vlastní další vývoj.

Právě tento okamžik tvoří bootstrap DarkFactory. Před vznikem základní pipeline musel člověk coding agenta spouštět a řídit přímo. Jakmile však pipeline dokáže sama reagovat na požadavky, spouštět produkční harness, uchovávat stav práce, vytvářet pracovní větve a vracet změnu k revizi, může být stejný proces použit i na repozitář DarkFactory samotný (35). Další funkce proto nemusí vznikat mimo systém, který mají rozšiřovat; mohou být přidávány postupně prostřednictvím stejné pipeline.

Tento postup propojuje principy z teoretické části do jednoho praktického vývojového procesu: specifikace vymezuje záměr, orchestrace řídí pořadí kroků, harness poskytuje agentní schopnosti, deterministické kontroly ověřují to, co lze ověřit programově, a člověk zůstává v rozhodovacích bodech procesu. Způsob vzniku DarkFactory sám o sobě nedokazuje vyšší kvalitu nebo spolehlivost agentem vytvořeného softwaru; ukazuje však, že stejný agentický workflow lze použít nejen jako předmět návrhu, ale také jako prostředek k jeho dalšímu rozšiřování.

4 Zjištění a diskuse

Výzkumná otázka se ptá, jaké architektonické a procesní principy se opakují v současném agentickém vývoji softwaru a jak jsou realizovány v systému DarkFactory. Z teoretické části vyplývá společný vzor: jazykový model není samostatným vývojovým systémem, ale pracuje uvnitř harnessu a širšího workflow, které mu poskytují kontext, nástroje, stav a oprávnění. Nad jednotlivými modelovými kroky pak stojí specifikace cíle, orchestrace procesu, programově proveditelné kontroly a explicitní rozhodovací body člověka (10; 20; 26). DarkFactory tyto prvky nespojuje do nové agentní smyčky, ale do repozitářového procesu, který používá existující coding-agent harnessy jako vykonávací vrstvu.

První podstatné zjištění případové studie je význam **trvalého stavu mimo model**. V DarkFactory není autoritativním záznamem práce přepis konverzace, ale GitHub: původní požadavek, interpretace a plán jsou zachyceny v issues a komentářích, implementace ve větví a commitech a výsledná změna v pull requestu. Jednotlivé agentní běhy proto mohou skončit a další běh na ně může navázat bez nutnosti zachovat celý předchozí kontext modelu (35). Praktická realizace tak odpovídá principu context engineering, podle něhož má dlouhodobý stav existovat odděleně od pracovního kontextového okna (20).

Druhé zjištění se týká **dělbý odpovědností**. Produkční harness zajišťuje agentní smyčku, nástroje a komunikaci s modelem, zatímco DarkFactory řídí, kdy se agent spustí, jaký stav obdrží, kam se jeho výstup uloží a co následuje dál. Deterministicky popsatelné činnosti, jako práce s Git větví, formátování, spuštění testů nebo změna stavu workflow, jsou prováděny programově; otevřené úlohy, například interpretace, plánování, implementace a modelová revize, jsou delegovány agentovi (35). Tento způsob kombinace modelového a programového řízení odpovídá současnému pojetí orchestrace, které oba přístupy kombinuje podle povahy jednotlivých kroků (26).

Třetím zjištěním je role **specifikace a lidských bran**. DarkFactory odděluje porozumění požadavku od implementačního plánu a před samotnou implementací vyžaduje lidské schválení obou kroků. Agent tak může samostatně zvolit konkrétní technický postup, ale začíná až poté, co člověk potvrdil záměr a navržený způsob realizace. Po implementaci se odpovědnost člověka vrací při přijetí výsledného pull requestu. Praktická část tím ukazuje human-in-the-loop nikoli jako průběžné ruční ovládání agenta, ale jako několik explicitních rozhodnutí kolem automatizované práce.

Čtvrté zjištění se týká **začlenění agenta do běžného softwarového workflow**. Výstupem DarkFactory není izolovaná odpověď modelu, ale větev, commity, dokumentace a pull request, které lze číst a revidovat běžnými nástroji. Podobný princip se objevuje i v produkční

praxi: Stripe popisuje interní coding agenty, jejichž změny končí pull requestem určeným k lidské revizi, a Meta popisuje agentní systém schopný dovést automatizovaný proces k pull requestu připravenému pro člověka (7; 8). DarkFactory je výrazně menší případ, ale používá stejnou základní hranici mezi autonomní prací a standardním integračním mechanismem repozitáře.

Samostatnou roli má **dokumentace**. Ve zkoumané revizi vzniká veřejná API dokumentace inline u kódu a dokumentační web se generuje z kanonických zdrojů repozitáře. Pro agentní systém je tento princip užitečný ve dvou směrech: agent pracuje se zdroji, které jsou verzovány společně s kódem, a člověk dostává čitelný pohled nad stejným systémem bez nutnosti udržovat druhou, ručně synchronizovanou dokumentaci (35). Dokumentace tak funguje jako rozhraní mezi strojově prováděným vývojem a lidskou kontrolou.

Případová studie zároveň ukázala omezení počáteční implementace. Automatická code review i kontrola souladu s plánem jsou samy modelové kroky, takže nepředstavují deterministický důkaz správnosti. Review smyčka je ohraničená a po vyčerpání povolených iterací může skončit bez explicitního čistého verdiktu. Testovací sady se po implementaci spustí a při neúspěchu dostane agent jeden opravný krok, ale po této opravě se ve zkoumané revizi stejná verifikace znovu neprovádí před vytvořením commitu. Stejně tak může nálezný označený jako mimo schválený plán doplnit rozsah jako `Plan Deviation` a `Scope Amendment` bez nového lidského schválení původního plánu (35). Tyto body nepopírají architektonický vzor, ale ukazují místa, kde jednoduchý bootstrap ještě neprosazuje své vlastní hranice tak důsledně, jak by mohl.

Dalším omezením je rozsah samotné případové studie. Zkoumán je jeden systém a jedna jeho revize; práce neměří obecnou úspěšnost modelů ani neporovnává jednotlivé harnessy. Nelze proto z výsledku odvodit, že stejné uspořádání bude stejně fungovat v každém repozitáři nebo s každým modelem. Přenositelné je především pozorování o skladbě procesu: schopnost modelu je jen jedna část výsledku a praktický agentický vývoj vzniká až propojením specifikace, kontextu, nástrojů, trvalého stavu, orchestrace, kontrol a lidského rozhodování.

DarkFactory současně ukazuje praktický význam tohoto spojení. První verze byla záměrně omezena tak, aby ji bylo možné vytvořit jako bootstrap pomocí coding agentů a následně stejnou pipeline použít k dalšímu rozšiřování vlastního systému. Přínosem případové studie proto není tvrzení, že agentem vytvořený software je automaticky kvalitnější, ale konkrétní ukázka, že agentické inženýrství lze převést z jednotlivého použití coding agenta do opakovatelného repozitářového procesu, ve kterém jsou jeho role a hranice explicitně navrženy.

5 Závěr

Cílem práce bylo popsat a systematizovat principy současného agentického inženýrství ve vývoji softwaru a na systému DarkFactory ukázat, co jejich propojení umožňuje v praxi. Tento cíl byl naplněn propojením teoretického popisu současných agentických systémů s inženýrskou případovou studií konkrétní GitHub-native softwarové továrny.

Teoretická část ukázala, že jazykový model je v agentickém vývoji pouze jednou součástí širšího systému. Praktické použití coding agentů stojí také na harnessu, práci s kontextem a nástroji, trvalém stavu, specifikaci cíle, orchestraci jednotlivých kroků a na rozlišení toho, co rozhoduje model, co může být provedeno programově a kde má rozhodnout člověk. Tyto prvky se v současných systémech objevují v různých podobách, ale dohromady tvoří opakující se vzor agentického softwarového vývoje.

DarkFactory tento vzor realizuje pomocí běžných komponent softwarového vývoje. GitHub slouží jako uživatelské rozhraní i trvalá stavová vrstva, GitHub Actions jako událostní a výpočetní prostředí, Docker jako izolace agentního běhu a produkční coding-agent harness jako vykonávací vrstva pro modelovou práci. Požadavek prochází interpretací a plánováním, následně implementací na pracovní větví, automatickou revizí a kontrolou souladu s plánem a nakonec lidským přijetím výsledného pull requestu. Dokumentace se přitom generuje z kanonických zdrojů repozitáře a propojuje konkrétní implementaci s člověkem, který systém řídí a reviduje.

Případová studie zároveň ukázala, že samotné spojení těchto částí nezaručuje správnost každého výsledku. Modelová review zůstává modelovým úsudkem, některé kontrolní kroky počáteční implementace nejsou ještě prosazeny jako tvrdé brány a změna rozsahu může ve zkoumané revizi pokračovat bez obnoveného lidského schválení. Závěry práce jsou proto omezeny na konkrétní architektonické a procesní principy pozorované v jednom systému, nikoli na obecné tvrzení o úspěšnosti coding agentů nebo jednotlivých modelů.

Význam DarkFactory spočívá také ve způsobu jejího vzniku. První verze byla záměrně navržena jako malý bootstrap, který bylo možné vytvořit pomocí komerčních coding agentů a který už následně dokáže stejné nástroje sám spouštět a řídit. Agentické inženýrství se tím v tomto případě neposouvá jen k automatizaci jednotlivého programátorského úkolu, ale k návrhu celého procesu, v němž lze opakovaně delegovat otevřenou inženýrskou práci a současně zachovat stav, kontrolu a lidské rozhodování.

Další rozvoj může navázat přímo na omezení popsaná v diskusi: zpřesnit deterministické brány, vyžadovat nové schválení při změně rozsahu a ověřit stejné principy na dalších repozitářích a s jinými harnessy. Práce tak uzavírá konkrétní případ DarkFactory, ale zároveň

ukazuje širší posun ve vývoji softwaru: s rostoucími schopnostmi coding agentů se část práce vývojáře přesouvá od provádění jednotlivých kroků k návrhu systému, který určuje, co agent ví, co smí udělat, jak se ověří jeho výstup a kdy musí rozhodnout člověk.

Seznam zdrojů

1. GITHUB. *GitHub Copilot code suggestions in your IDE* [online]. 2026 [vid. 2026-09-22]. Dostupné z: <https://docs.github.com/en/copilot/concepts/completions/code-suggestions>
2. KHURANA, Sonam a Anusha VADDI. ChatGPT From the Perspective of an Academic Oral and Maxillofacial Radiologist. *Cureus* [online]. 2023, **15**(6). Dostupné z: [doi:10.7759/cureus.40053](https://doi.org/10.7759/cureus.40053). Obrázek 4. Screenshot z bezplatné verze ChatGPT, převzato jako ilustrace
3. GITHUB. *Use GitHub Copilot agents* [online]. 2026 [vid. 2026-09-22]. Dostupné z: <https://docs.github.com/en/copilot/how-tos/use-copilot-agents>
4. OPENAI. *Introducing Codex* [online]. 2025 [vid. 2026-09-22]. Dostupné z: <https://openai.com/index/introducing-codex/>
5. OPENAI. *Introducing the Codex app* [online]. 2026 [vid. 2026-09-22]. Dostupné z: <https://openai.com/index/introducing-the-codex-app/>
6. HILLEBRANDT, Finn. *How Many People Use AI? Facts & Figures 2026* [online]. 2026 [vid. 2026-09-22]. Dostupné z: <https://www.gradually.ai/en/how-many-people-use-ai/>. Published 2026-08-23
7. STRIPE. *Developer keynote: Advancing developer craft* [online]. 2026 [vid. 2026-09-30]. Dostupné z: <https://stripe.com/sessions/2026/developer-keynote>. Stripe Sessions 2026
8. META. *Capacity Efficiency at Meta: How Unified AI Agents Optimize Performance at Hyperscale* [online]. 2026 [vid. 2026-09-30]. Dostupné z: <https://engineering.fb.com/2026/04/16/developer-tools/capacity-efficiency-at-meta-how-unified-ai-agents-optimize-performance-at-hyperscale/>
9. MERRIAM-WEBSTER. *Agentic: Definition & Meaning* [online]. 2026 [vid. 2026-09-26]. Dostupné z: <https://www.merriam-webster.com/dictionary/agentic>. Původ slova: agent + -ic, poprvé doloženo 1966
10. TRIVEDY, Vivek. *The Anatomy of an Agent Harness* [online]. 2026 [vid. 2026-09-27]. Dostupné z: <https://www.langchain.com/blog/the-anatomy-of-an-agent-harness>. LangChain, 10. března 2026
11. VASWANI, Ashish, Noam SHAZEER, Niki PARMAR, Jakob USZKOREIT, Llion JONES, Aidan N. GOMEZ, Łukasz KAISER a Illia POLOSUKHIN. Attention Is All You Need. *ar-*

- Xiv preprint arXiv:1706.03762* [online]. 2017. Dostupné z: <https://arxiv.org/abs/1706.03762>
12. MIKOLOV, Tomas, Wen-tau YIH a Geoffrey ZWEIG. Linguistic Regularities in Continuous Space Word Representations. In: *Proceedings of the 2013 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies* [online]. 2013, s. 746–751. Dostupné z: <https://aclanthology.org/N13-1090/>
 13. YAO, Shunyu, Jeffrey ZHAO, Dian YU, Nan DU, Izhak SHAFRAN, Karthik NARASIMHAN a Yuan CAO. ReAct: Synergizing Reasoning and Acting in Language Models. *arXiv preprint arXiv:2210.03629* [online]. 2022. Dostupné z: <https://arxiv.org/abs/2210.03629>
 14. ANTHROPIC. *Building effective agents* [online]. 2024 [vid. 2026-09-23]. Dostupné z: <https://www.anthropic.com/engineering/building-effective-agents>
 15. AGENT SKILLS. *Agent Skills: an open format for extending AI agents* [online]. 2025 [vid. 2026-09-22]. Dostupné z: <https://agentskills.io/home>
 16. OPENAI. *Lifecycle / OpenAI Agents SDK* [online]. 2026 [vid. 2026-09-22]. Dostupné z: <https://openai.github.io/openai-agents-python/ref/lifecycle/>
 17. MODEL CONTEXT PROTOCOL. *Model Context Protocol 2026-07-28 specification: server features* [online]. 2026 [vid. 2026-09-22]. Dostupné z: <https://modelcontextprotocol.io/specification/2026-07-28/server>
 18. AGENTIC AI FOUNDATION. *AGENTS.md* [online]. 2026 [vid. 2026-09-23]. Dostupné z: <https://agents.md/>
 19. LIU, Nelson F., Kevin LIN, John HEWITT, Ashwin PARANJAPE, Michele BEVILACQUA, Fabio PETRONI a Percy LIANG. Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics* [online]. 2024, **12**, 157–173. Dostupné z: <https://arxiv.org/abs/2307.03172>
 20. ANTHROPIC. *Effective context engineering for AI agents* [online]. 2025 [vid. 2026-09-22]. Dostupné z: <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>
 21. ALENEZI, Mamdouh. Educating the Agentic Engineer: Curricula, Collaboration, and Continuous Learning in the AI Era. *arXiv preprint arXiv:2607.29610* [online]. 2026 [vid. 2026-09-26]. Dostupné z: <https://arxiv.org/abs/2607.29610>

22. OPENAI. *Prompt engineering* [online]. 2026 [vid. 2026-09-22]. Dostupné z: <https://platform.openai.com/docs/guides/prompt-engineering>
23. ANTHROPIC. *Harness design for long-running application development* [online]. 2026 [vid. 2026-09-21]. Dostupné z: <https://www.anthropic.com/engineering/harness-design-long-running-apps>
24. OPENAI. *Sandbox agents: Concepts | OpenAI Agents SDK* [online]. 2026 [vid. 2026-09-22]. Dostupné z: <https://openai.github.io/openai-agents-js/guides/sandbox-agents/concepts/>
25. OPENAI. *Using Goals in Codex: Persistent Objectives for Long-Running Work* [online]. 2026 [vid. 2026-09-27]. Dostupné z: https://developers.openai.com/cookbook/examples/codex/using_goals_in_codex
26. OPENAI. *Agent orchestration | OpenAI Agents SDK* [online]. 2026 [vid. 2026-09-21]. Dostupné z: https://openai.github.io/openai-agents-python/multi_agent/
27. IEEE. *IEEE Guide for Software Requirements Specifications* [online]. 1984 [vid. 2026-09-30]. Dostupné z: <https://standards.ieee.org/ieee/830/1220/>. IEEE 830-1984
28. ISO/IEC/IEEE. *ISO/IEC/IEEE 29148:2018 — Systems and software engineering — Life cycle processes — Requirements engineering* [online]. 2018 [vid. 2026-09-30]. Dostupné z: <https://www.iso.org/standard/72089.html>
29. GALLARDO, Luis. *Beyond Cursor: My Experience with Claude Code and Sequential Thinking* [online]. 2025 [vid. 2026-09-28]. Dostupné z: <https://lgallardo.com/2025/07/09/beyond-cursor-claude-code-sequential-thinking/>. Obrázek s plánem, který Claude Code předkládá ke schválení. Převzatá ilustrace rozhraní, nikoli záznam běhu popsané pipeline
30. GOOGLE. *Antigravity CLI* [online]. 2026 [vid. 2026-09-27]. Dostupné z: <https://antigravity.google/product/antigravity-cli>. Sekce Subagents functionalities. Převzatá ilustrace rozhraní, nikoli záznam běhu
31. ANTHROPIC. *Orchestrate subagents at scale with dynamic workflows* [online]. 2026 [vid. 2026-09-27]. Dostupné z: <https://code.claude.com/docs/en/workflows>. Sekce Dynamic workflows. Převzatá ilustrace rozhraní, nikoli záznam běhu

32. HUNTLEY, Geoffrey. *Ralph Wiggum as a „software engineer“* [online]. 2025 [vid. 2026-09-27]. Dostupné z: <https://ghuntley.com/ralph/>. 14. července 2025. Původní text, který techniku pojmenoval
33. ANTHROPIC. *Keep Claude working toward a goal* [online]. 2026 [vid. 2026-09-27]. Dostupné z: <https://code.claude.com/docs/en/goal>
34. NATO. *Software Engineering: Report of a Conference Sponsored by the NATO Science Committee, Garmisch, Germany, 7–11 October 1968* [online]. Brussels: NATO, Scientific Affairs Division, 1969 [vid. 2026-09-27]. Dostupné z: <http://homepages.cs.ncl.ac.uk/brian.randell/NATO/nato1968.PDF>. 231 s. Zpráva o softwarové krizi; stažitelný přepis z roku 2001 má 136 stran, číslování citací se vztahuje k původnímu vydání
35. MARIUS, Patrik. *DarkFactory revize d576ec8f* [online]. 2026 [vid. 2026-09-26]. Dostupné z: <https://github.com/marius-patrik/DarkFactory/tree/d576ec8fdb78a654bffc9d0da165aa81a97b673>. Poslední revize, v níž ještě pipeline volá produkční harne-sy

Seznam součástí textu

Obrázek 1	Doplňování kódu přímo v editoru: model navrhuje pokračování řádku, které člověk přijme nebo odmítne (1).....	6
Obrázek 2	Chatbot odpovídá textem a práci nechává člověku: místo aby snímek posoudil, přizná, že obrázky neumí, a je to uživatel, kdo musí dojít k závěru. Snímek z bezplatné verze ChatGPT, převzato z (2).....	7
Obrázek 3	Odhad rozdělení uživatelů generativní AI podle typu (6).....	8
Obrázek 4	Od jazykového modelu k agentickému systému. Jazykový model je jednou ze čtyř součástí harnessu; repozitář, oprávnění ani trvalý stav mu nedává samotný, ale vrstva, která jeho návrh převádí na akci s dohledatelným výsledkem. .11	
Obrázek 5	Ilustrace sémantického vztahu mezi vektorovými reprezentacemi slov <i>král</i> , <i>královna</i> , <i>muž</i> a <i>žena</i> (12).....	12
Obrázek 6	Vzor ReAct: myšlenku navrhuje model, akci provádí harness a pozorování mu vrátí. Každý krok se připojí do kontextu, takže smyčka pokračuje na tom, co už model viděl. Končí odpovědí místo dalšího požadavku na nástroj (13). . . .	13
Obrázek 7	Plán, který model navrhne, než se začne psát kód: seznam kroků podle souborů, které se mění, a otázka, zda se má pokračovat. Znění plánu je tak oddělené od jeho provádění — je to rozhodnutí, které člověk schvaluje nebo odmítne, nikoli popis práce, která už běží. Snímek z Claude Code, převzato z (29). . . .	16
Obrázek 8	Rozhraní Google Antigravity CLI, tedy harnessu agenta běžícího v terminálu: koordinátor definuje tři specializované subagenty a spouští je souběžně; každý běží vlastním kontextem (30).....	18
Obrázek 9	Postup práce dynamického workflow v Claude Code: vlevo fáze plánu se stavem, vpravo rozvinutá fáze <i>CodeReview</i> se 16 agenty, u každého model, spotřebované tokeny, počet nástrojů a doba běhu. Snímek pochází z komunitního příspěvku; rozhraní i jeho chování popisuje dokumentace (31).....	19
Obrázek 10	Cíl, proti němuž agent postupuje, a jeho splnění: po dokončení dílčího kroku se stav porovná s cílem a běh v tomto případě končí, včetně doby, kterou zabral. Snímek pochází z komunitního příspěvku; funkci popisuje dokumentace (25).....	19
Obrázek 11	Architektura DarkFactory: GitHub poskytuje události a trvalý vývojový stav, GitHub Actions výpočetní prostředí, Docker izolaci běhu a pythonovský runner převádí událost na agentní krok. Produkční harness zajišťuje model, nástroje a agentní smyčku (35).....	26

Tabulka 1	Pole předlohy issue pro uživatelský požadavek: tři jsou povinná a tři volitelná (35).....	28
Obrázek 12	Průchod požadavku DarkFactory: interpretace a plánování jsou odděleny lidskými bránami, implementace probíhá na pracovní větvi a draft pull request prochází automatickou revizí a kontrolou souladu s plánem před lidským přijetím výsledku (35).....	29